Parser Stage 实验报告

于越洋

1 实验内容

Parser Step

我完成了 `my_parser.py` 中的下列函数

```
p_relational p_logical_and p_assignment p_expression
p_statement p_declaration p_block p_return p_type
```

`p_relational`

用于处理关系表达式，包括大于、小于、大于等于、小于等于四种运算。函数将首先解析一个 `additive` 运算语句作为左操作数，再通过 `lookahead` 获取运算符，再解析一个 `additive` 运算语句作为右操作数，构建二元运算节点，再不断重复获取运算符、解析右操作数、构建节点的过程，以处理多个关系运算连续的情况。在 `next` 不再是关系运算符时返回节点。

`p_logical_and`

用于处理逻辑与表达式。函数将首先解析一个 `equality` 运算语句作为左操作数，再通过 `lookahead` 获取运算符，再解析一个 `equality` 运算语句作为右操作数，构建二元运算节点，再不断重复获取运算符、解析右操作数、构建节点的过程，以处理多个关系运算连续的情况。在 `next` 不再是逻辑与运算符时返回节点。

`p_assignment`

用于处理赋值表达式，函数首先解析可能存在的条件语句获得一个标识符，再判断 `next` 是否为 `=`，若是则解析 `=` 运算符，再解析一个 `expression` 语句作为右操作数，使用左侧的标识符和右操作数来构建赋值运算节点并返回。

`p_expression`

解析一个 `assignment` 赋值语句并返回。

`p_statement`

用于解析语句，包括 `return` 语句、以分号结尾的（可能存在的）表达式、语句块，所以 `statement` 的 First 集合包括 `Return`， `Semi`， `LBrace` 和 `expression` 的 First 集合。
如果 `next` 是 `expression` 的 First 集合中的字符，则解析一个 `expression` 表达式和分号，返回解析的 `expression` 节点。

如果 `next` 是分号，则直接解析分号并返回空节点。
如果 `next` 是左大括号，则解析一个 `block` 语句块并返回节点。
如果 `next` 是 `Return`，则解析一个 `return` 语句并返回节点。

`p_declaration`

用于解析声明语句，我补充了声明时赋值的部分，即若声明的标识符后面跟随着一个 `=`，则解析 `=` 和一个 `expression` 表达式，并将声明语句节点的 `init_expr` 设置为该表达式结果。

`p_block`

用于解析语句块。首先创建一个 `block` 节点，解析一个左大括号，接下来在解析到右大括号之前，不断调用 `p_block_item` 函数逐条解析语句并加入 `block` 节点的 `children` 列表中。
对于 `block_item`，如果 `next` 是 `statement` 的 First 集合中的字符，则解析一个 `statement` 语句并返回对应节点；如果 `next` 是 `declaration` 的 First 集合中的字符，则解析一个 `declaration` 语句和分号并返回 `declaration` 节点。

`p_return`

用于解析返回语句，首先解析一个 `Return` 关键字，然后解析一个 `expression` 表达式，再解析一个分号，最后使用解析的 `expression` 表达式构建 `Return` 节点返回。

`p_type`

解析一个 `Int` 关键字并返回 `TInt` 类型节点。

2 思考题

Question 1

> 在框架里我们使用 EBNF 处理了 `additive` 的产生式。请使用课上学习的消除左递归、消除左公因子的方法，将其转换为不含左递归的 LL(1) 文法。（不考虑后续 multiplicative 的产生式）

```
additive : additive '+' multiplicative
         | additive '-' multiplicative
         | multiplicative
```

Answer：转换后的文法为

```
additive : multiplicative additive'
additive' : '+' multiplicative additive'
          | '-' multiplicative additive'
          | ε
```

Question 2

> 对于我们的程序框架，在自顶向下语法分析的过程中，如果出现一个语法错误，可以进行**错误恢复**以继续解析，从而继续解析程序中后续的语法单元。请尝试举出一个出错程序的例子，结合我们的程序框架，描述你心目中的错误恢复机制对这个例子，怎样越过出错的位置继续解析。（注意目前框架里是没有错误恢复机制的。）

Answer：使用如下出错程序

```
int main() {
    int a = 1;
    int b = 2  // 缺少分号
    int c = 3;
    return a;
}
```

当前解析器的行为是在 `lookahead` 函数中抛出一个 `DecafSyntaxError` 异常。
我心目中的错误恢复机制，可以在遇到错误时跳过出错的 tokens，直到遇到同步符号，继续进行解析。对于上面的出错程序，可以设定同步符号集为包含 `Int` 关键字、`Return` 关键字和 `LBrace` 的集合，当解析到出错位置时，记录缺少分号错误，并继续读取，读到下一行的 `Int` 关键字，认为遇到了同步符号，继续正常解析后面的语句。